

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Системы искусственного интеллекта

Лабораторная работа №1

База знаний на Prolog и онтология в Protégé

Подбор игровых персонажей для выполнения заданий

Выполнил:
Чэнь Хаолинъ Р3316 407960
Преподаватель:
Болдырева Елена Александровна

г. Санкт-Петербург, 2026

Содержание

1	Введение	2
2	Анализ требований	2
2.1	Функциональные требования	2
2.2	Ограничения модели	2
3	Изучение основных концепций и инструментов	3
3.1	База знаний и логический вывод	3
3.2	Онтология и выбранные средства	3
4	Реализация системы искусственного интеллекта	3
4.1	Объекты и факты	3
4.2	Семь правил Prolog	4
4.3	Перенос в онтологию	4
4.4	Отрицание и границы эквивалентности	5
4.5	Воспроизводимая проверка и работа в Protégé	5
5	Оценка и интерпретация результатов	6
5.1	Запросы к базе знаний	6
5.2	Запросы к онтологии и сравнение	6
5.3	Интерпретация и развитие	7
6	Заключение	7
	Источники	7
A	Файлы проекта и воспроизведение	8

1. Введение

Цель работы — создать базу знаний об игровых персонажах, реализовать логические запросы на Prolog и перенести факты и отношения в онтологию, пригодную для работы в Protégé. Предметная область представляет собой небольшой вымышленный мир RPG: персонажи владеют оружием, обладают навыками, состоят в гильдиях и могут подходить для определённых заданий.

Практическая задача состоит в получении объяснимого ответа на вопрос: какой персонаж имеет оружие и навык, необходимый для задания? Например, для посещения башни требуется магия, поэтому при заданных фактах подходит Celia. Ответ определяется явно заданными условиями, а не статистическим прогнозом.

Работа охватывает обе части лабораторной 1: Prolog и онтологическое представление. Структура отчёта соответствует шаблону модуля 1. Диалоговая рекомендательная программа из лабораторной 2 относится к дальнейшему развитию и не входит в реализованный объём этой работы.

2. Анализ требований

2.1. Функциональные требования

База должна хранить типы объектов, роли персонажей, отношения владения, навыки и требования заданий. Необходимо поддерживать поиск с переменными, конъюнкцию, дизъюнкцию, отрицание и вывод по правилам. Результат подбора должен иметь проверяемое объяснение: предмет типа Weapon и совпадающий навык.

Для онтологии требуются классы, иерархия, свойства с областями определения и значений, индивиды и проверка выводимых знаний. Семантические различия Prolog и OWL должны быть описаны явно.

Требование задания	Реализация	Проверка
Не менее 20 унарных фактов	24	Разбор исходника
От 10 до 15 бинарных фактов	15	Разбор исходника
От 5 до 7 правил	7	knowledge.pl
Разнообразные запросы	15 тестов	SWI-Prolog
Комментарии к фактам и правилам	На русском языке	Исходный код
Перенос базы в онтологию	OWL и SWRL	Pellet, SPARQL
Сравнение двух представлений	7 сопоставлений	Проверка множеств

Таблица 1: Соответствие количественным и функциональным требованиям

2.2. Ограничения модели

Данные являются учебными и вымышленными. Готовность к заданию означает только наличие оружия и подходящего навыка: уровень, здоровье, сложность, стоимость и вероятность победы не моделируются. Состояние Cursed рассматривается отдельно и не включено в правило готовности. Персонаж может иметь несколько ролей: несовместимость ролей не постулируется.

В базе хранятся шесть персонажей, три вида оружия, три задания, три навыка и две гильдии. Повторное использование **sword** означает владение видом оружия, а не одним уникальным физическим мечом. Имена являются стабильными идентификаторами объектов.

3. Изучение основных концепций и инструментов

3.1. База знаний и логический вывод

Факт `warrior(aria)` задаёт свойство объекта, а `owns(aria,sword)` — отношение двух объектов. Правило задаёт условие вывода нового утверждения. Prolog использует унификацию и поиск с возвратом: переменная связывается с подходящим значением, а при запросе альтернатив перечисляются остальные решения.

В записи правила запятая обозначает «и», точка с запятой — «или», оператор `\+` — отрицание как неуспех доказательства. В `safe_character(X)` сначала выполняется `character(X)`, поэтому отрицание проверяется для конкретного известного персонажа. Перестановка этих целей при свободной переменной изменила бы поведение программы.

3.2. Онтология и выбранные средства

OWL описывает классы, свойства и ограничения, а механизм рассуждений выводит дополнительные типы и отношения. Protégé служит редактором и средством исследования такой модели. Онтология сохранена в RDF/XML и не требует внешних импортов. Для четырёх унарных правил используются определения классов OWL; для трёх правил, связывающих переменные через общий навык или гильдию, используются SWRL-правила.

Проверка Prolog выполняется в настоящем SWI-Prolog, поставляемом в пакете `swipl-wasm`, под Node.js. Скрипт `run_prolog.mjs` загружает исходник и сравнивает результаты с эталонами. Онтология строится через Owlready2 0.48, проверяется Pellet 2.3.1 под Java 17, а SPARQL-запросы выполняются RDFLib 7.6.0 по графу с материализованными выводами. SPARQL сам по себе не заменяет OWL/SWRL-рассуждение.

4. Реализация системы искусственного интеллекта

4.1. Объекты и факты

Унарные факты распределены следующим образом: 6 фактов `Character`, 6 фактов ролей, 3 факта `Weapon`, 3 факта `Quest`, 3 факта `Skill`, 2 факта `Guild` и 1 факт `Cursed`. Итого: 24 факта о 17 именованных объектах. Число фактов не совпадает с числом объектов, поскольку один персонаж имеет несколько свойств.

Персонаж	Роль	Оружие	Навык	Гильдия
aria	warrior	sword	melee	sun
borin	warrior	sword	—	moon
celia	mage	staff	magic	sun
darin	healer	staff	—	—
elin	rogue	bow	—	—
finn	ranger	bow	tracking	—

Таблица 2: Исходные сведения о персонажах; прочерк означает отсутствие факта

Для Borin указан факт `cursed(borin)`. Задания требуют следующих навыков: `ruins` — `melee`, `forest` — `tracking`, `tower` — `magic`. Бинарные факты включают 6 отношений владения, 3 навыка, 3 требования и 3 отношения членства: всего 15.

4.2. Семь правил Prolog

В листинге приведена полная логика вывода. Факты и русские комментарии находятся в приложенном файле `knowledge.pl`.

`knowledge.pl` - правила

```
1 combatant(X) :-  
2     character(X), (warrior(X); mage(X)).  
3 scout(X) :-  
4     character(X), (rogue(X); ranger(X)).  
5 armed(X) :-  
6     character(X), owns(X,W), weapon(W).  
7 skilled_for(X,Q) :-  
8     character(X), quest(Q),  
9     has_skill(X,S), requires(Q,S).  
10 ready_for(X,Q) :-  
11     armed(X), skilled_for(X,Q).  
12 guildmate(X,Y) :-  
13     character(X), character(Y),  
14     member_of(X,G), member_of(Y,G), dif(X,Y).  
15 safe_character(X) :-  
16     character(X), \+ cursed(X).
```

Первые два правила классифицируют персонажей по альтернативным ролям. Третье проверяет наличие оружия. Четвёртое соединяет отношения по общей переменной навыка `S`; пятое объединяет результаты двух правил. Шестое находит разных членов одной гильдии и исключает пару объекта с самим собой. Седьмое реализует отрицание в границах известных персонажей.

4.3. Перенос в онтологию

Унарные предикаты перенесены в классы, факты — в принадлежность индивидов этим классам. Бинарные предикаты перенесены в объектные свойства. Всего определены 15 классов, 7 объектных свойств и 17 предметных индивидов. Три из семи свойств предназначены для выводимых отношений.

Свойство	Domain	Range
<code>owns</code>	Character	Weapon
<code>hasSkill</code>	Character	Skill
<code>requiresSkill</code>	Quest	Skill
<code>memberOf</code>	Character	Guild
<code>skilledFor</code>	Character	Quest
<code>readyFor</code>	Character	Quest
<code>guildmate</code>	Character	Character

Таблица 3: Объектные свойства онтологии

Классы `Character`, `Weapon`, `Quest`, `Skill` и `Guild` попарно несовместимы. `Warrior`, `Mage`, `Healer`, `Rogue`, `Ranger` и `Cursed` являются подклассами `Character`. Области `Domain` и `Range` используются для логического вывода типов, а не как процедура отклонения некорректных строк данных.

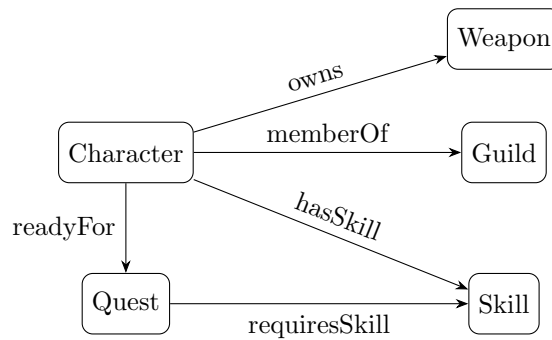


Рис. 1: Схема основных отношений модели; схема построена по онтологии, не является снимком интерфейса Protégé

Определения классов в синтаксисе Manchester имеют вид:

```

1 Combatant EquivalentTo Character and (Warrior or Mage)
2 Scout EquivalentTo Character and (Rogue or Ranger)
3 Armed EquivalentTo Character and (owns some Weapon)
4 SafeCharacter EquivalentTo Character and not Cursed

```

Правила с общими переменными представлены в SWRL:

```

1 Character(?x), Quest(?q), hasSkill(?x,?s),
2 requiresSkill(?q,?s) -> skilledFor(?x,?q)
3
4 Armed(?x), skilledFor(?x,?q) -> readyFor(?x,?q)
5
6 Character(?x), Character(?y), memberOf(?x,?g),
7 memberOf(?y,?g), differentFrom(?x,?y)
8 -> guildmate(?x,?y)

```

Для всех предметных индивидов задано AllDifferent. Это необходимо, поскольку в OWL разные имена не гарантируют различие объектов; в Prolog различные атомы различаются непосредственно.

4.4. Отрицание и границы эквивалентности

В Prolog отсутствие доказательства `cursed(aria)` позволяет получить `safe_character(aria)`. В OWL при открытом мире отсутствие принадлежности Cursed не доказывает принадлежность дополнению этого класса. Поэтому для пяти непроклятых персонажей текущего снимка явно задан тип `not Cursed`. Это дополнительное предположение о полноте сведений о проклятии, а не автоматическое следствие отсутствия факта.

При добавлении нового персонажа без информации о проклятии Prolog сочтёт его безопасным, а OWL оставит классификацию неопределённой. При добавлении `cursed(aria)` необходимо пересмотреть отрицательное OWL-утверждение: сохранение обоих утверждений даст противоречие. Следовательно, равенство результатов относится к текущему снимку и оговорённым предположениям, а не ко всем возможным расширениям базы.

4.5. Воспроизводимая проверка и работа в Protégé

Сначала выполняется `node run_prolog.mjs`, затем `python build_ontology.py`. Второй скрипт извлекает исходные факты из Prolog, сохраняет `game_ontology.owl`, запускает Pellet и создаёт отдельный `results/game_inferred.owl`. После этого проверяются SPARQL-запросы и сопоставляются результаты семи правил с результатами SWI-Prolog.

Для демонстрации в Protégé следует открыть исходный OWL-файл через File / Open, посмотреть

вкладки `Classes`, `Object properties` и `Individuals`. В дереве `Character` доступны роли и производные классы. Визуализацию иерархии можно открыть через `OntoGraf` при наличии этого представления. Для вывода SWRL-отношений требуется поддерживающий их reasoner, например `Pellet`; простой просмотр исходных RDF-троек такие отношения не вычисляет.

Материализованный файл можно открыть отдельно, чтобы увидеть уже проверенные отношения, например `aria readyFor ruins`. В DL Query выражение `Character and (readyFor value ruins)` выбирает `Aria`, а `Combatant and SafeCharacter` — `Aria` и `Celia` при доступном выводе или загруженных материализованных типах. Автоматические результаты в этом отчёте получены через `Pellet` и `RDFLib`; ручная сессия и снимки экрана `Protégé` не использовались как подтверждение тестов.

5. Оценка и интерпретация результатов

5.1. Запросы к базе знаний

В таблице показаны фактически полученные результаты. Пустое множество означает отсутствие решений; для полностью конкретного запроса это соответствует `false`. Для положительного конкретного запроса указано `true`.

Запрос Prolog	Результат
<code>character(aria)</code>	<code>true</code>
<code>character(dragon)</code>	<code>false</code>
<code>warrior(X)</code>	<code>aria, borin</code>
<code>owns(X,bow)</code>	<code>elin, finn</code>
<code>combatant(X)</code>	<code>aria, borin, celia</code>
<code>scout(X)</code>	<code>elin, finn</code>
<code>armed(X)</code>	<code>aria, borin, celia, darin, elin, finn</code>
<code>skilled_for(X,tower)</code>	<code>celia</code>
<code>ready_for(X,Q)</code>	<code>(aria, ruins), (celia, tower), (finn, forest)</code>
<code>guildmate(X,Y)</code>	<code>(aria, celia), (celia, aria)</code>
<code>safe_character(X)</code>	<code>aria, celia, darin, elin, finn</code>
<code>ready_for(borin,ruins)</code>	<code>false</code>
<code>guildmate(aria,aria)</code>	<code>false</code>
<code>combatant(X), safe_character(X)</code>	<code>aria, celia</code>
<code>safe_character(dragon)</code>	<code>false</code>

Таблица 4: Результаты 15 тестов SWI-Prolog

Контрпример `Borin` показывает, что наличия меча недостаточно: у него не задан навык `melee`. `Elin` и `Darin` также вооружены, но не имеют подходящих навыков в текущей базе. Пары членов гильдии симметричны вследствие тела правила, хотя симметричность свойства отдельно не объявлена. Проверка пары `Aria` с собой подтверждает действие ограничения различия.

5.2. Запросы к онтологии и сравнение

Пример SPARQL-запроса по материализованному графу:

```
1 PREFIX : <https://example.org/rpg-lab1#>
2 SELECT ?x ?q WHERE { ?x :readyFor ?q }
```

Получены пары `aria-ruins`, `celia-tower` и `finn-forest`. Порядок выдачи без `ORDER BY` не гарантируется, поэтому при сопоставлении результаты нормализуются и сортируются.

Для класса безопасных персонажей используется положительный запрос к выведенному типу:

```

1 PREFIX : <https://example.org/rpg-lab1#>
2 SELECT ?x WHERE { ?x a :SafeCharacter }

```

Подмена этого запроса фильтром NOT EXISTS по Cursed означала бы проверку отсутствия тройки в графе, а не доказательство OWL-отрицания. В реализации такая подмена не используется.

Проверка	Количество	Успешно
Запросы SWI-Prolog	15	15
Запросы SPARQL после Pellet	9	9
Сопоставления Prolog и OWL	7	7

Таблица 5: Итог автоматической проверки

Pellet завершил рассуждение без обнаружения противоречия. Девять запросов охватывают четыре производных класса, три отношения и два отрицательных случая: отсутствие задания для Borin и отсутствие отношения guildmate к самому себе. JSON-протоколы содержат сами запросы, ожидаемые и фактические результаты. Проверки подтверждают текущий набор сценариев, но не являются доказательством корректности любых будущих изменений.

5.3. Интерпретация и развитие

Все количественные требования выполнены. Для учебной базы Prolog удобен компактностью правил и прямым интерактивным поиском. Онтология добавляет явную структуру типов, описание свойств и контроль логической согласованности, но требует осторожного обращения с отрицанием и различием индивидов.

На данном небольшом наборе не проводилось измерение производительности; утверждения о скорости или масштабируемости не делаются. Следующие шаги: добавить уровни и ограничения заданий, уточнить полноту сведений о состояниях персонажей, выдавать цепочку объяснения результата и реализовать диалоговый интерфейс лабораторной 2. При росте базы полезны отдельные проверки качества исходных данных и регрессионные тесты.

6. Заключение

Разработана прокомментированная база знаний из 24 унарных фактов, 15 бинарных фактов и 7 правил. Выполнены запросы разных типов, включая поиск с переменными, логические операции, составной вывод и отрицательные случаи. Подготовлена OWL/SWRL-онтология с иерархией классов, областями свойств и явными предположениями, необходимыми для сравнения с Prolog.

Получены три объяснимые пары персонажей и заданий; результаты семи правил согласуются в двух представлениях при фиксированном снимке знаний. Модель может служить основой для помощника игрока, проверки игровых данных или учебной системы поддержки принятия решений. Основной вывод работы: перенос знаний требует согласования семантики, а не только замены синтаксиса фактов.

Источники

1. Задание купца AI System. <https://sunnysubmarines.notion.site/AI-System-a559a46cddc44363bdf27b77e10b7d85>.
2. Шаблон отчёта по модулю 1. <https://sunnysubmarines.notion.site/1-ea9a418d514b4baa8f7b4fc4c3ee176e>.

3. SWI-Prolog. Официальный сайт и документация. <https://www.swi-prolog.org/>.
4. Protégé. Официальный сайт. <https://protege.stanford.edu/>.
5. W3C. OWL 2 Structural Specification and Functional-Style Syntax. <https://www.w3.org/TR/owl-syntax/>.

А. Файлы проекта и воспроизведение

knowledge.pl

Все факты, правила и комментарии на русском языке.

run_prolog.mjs

Исполнение 15 запросов в SWI-Prolog и проверка ответов.

build_ontology.py

Перенос фактов, OWL/SWRL-аксиомы, запуск Pellet, SPARQL и сопоставление результатов.

game_ontology.owl

Исходная онтология для открытия в Protégé.

queries.sparql

Девять отдельных запросов к графу с выводами.

results/

JSON-протоколы и онтология с материализованными выводами.

report/main.tex, title.tex, 1.tex

Исходный текст настоящего отчёта.

Команды выполняются из каталога Lab1. Требуется Node.js с npm, Python и Java. Зависимости устанавливаются из файлов проекта:

```
1 npm ci
2 python -m pip install -r requirements.txt
3 node run_prolog.mjs
4 python build_ontology.py
```

Альтернативно Prolog можно запустить в настольном SWI-Prolog, загрузив **knowledge.pl**, и вводить запросы из таблицы вручную. Для сборки отчёта из каталога **report**:

```
1 xelatex main.tex
2 xelatex main.tex
3 % or:
4 tectonic main.tex
```

Оформление соответствует предоставленному образцу: формат A4, кегль 11 pt, поля 2 см слева, справа и сверху и 3 см снизу, шрифты CMU Serif и CMU Typewriter Text, отдельные файлы титульного листа и основной части, оглавление и нумерованные листинги. Файлы шрифтов с лицензией включены в каталог **fonts**. Данные студента находятся в **title.tex**, имя преподавателя задаётся командой **TeacherName** в **main.tex**.